Computational Control

Lecture Handout

March 23, 2026

Contents

1 Dynamic Programming and LQR

1.1 Introduction

In general we formulate the optimization control task as

$$\min_{\mathbf{u}\in\mathcal{U}} J(\mathbf{u}) \tag{1}$$

where $\mathcal{U} = \tilde{\mathcal{U}} \times \{0, 1, \dots, \mathrm{T}\}$ is the set of the feasible inputs and $J(\mathbf{u})$ is generally the sum of some *stage costs* and a terminal cost/constraint.
The issue is that generally 1 is an intractable optimization problem. This is due to several factors:

- the dimension of $\mathbf{u}$ is too big;
- the cost function requires an accurate model of the system;
- the optimization problem is **non-convex**;
- In presence of **disturbances** and **uncertainity** the open-loop nature of the task makes it useless

In this context, Dynamic Programming (DP) is a powerfull tool whenever we can decompose the problem. In particular when the problem allows for a **Markovian representation**:

Key assumptions: Markovian representation

The following hypothesis are crucial in order for DP to be meaningfull:

1. There exists a **Markovian State**[a] that evolves according to some dynamics
$$x_{t+1} = f_t(x_t, u_t)^{[b]}$$
2. The initial condition x_0 is known.
3. The cost is additive over time, i.e.
$$J = \sum_t g_t(x_t, u_t)$$

[a] i.e. $X_{t+1} \perp X_{1:t-1} | X_t$
[b] discrete time dynamical system representation. Notice f_t can change with time

In this context a powerfull result is given by **Bellman's principle**. Considering the following setup:

$$\begin{aligned} \min_{\mathbf{u},\mathbf{x}} \quad & \sum_{t=0}^{T} g_t(x_t, u_t) \\ \text{subject to} \quad & x_{t+1} = f_t(x_t, u_t) \\ & x_0 = X_0 \\ & x_t \in \mathcal{X}_t \ \forall t \\ & u_t \in \mathcal{U}_t \ \forall t. \end{aligned}$$

a very powerfull result on which relies DP is **Bellman's optimality principle**.

Bellman's optimality principle

An optimal policy has the property that whatever the initial state and the initial decisions are, the remaining decisions must consititute an optimal policy with regard to the state resulting from the first decision.

In a intuitive way we can say that any tail of an optimal trajectory is optimal too.

$$\begin{aligned} V_0(X_0) &= \min_{u_0} \left\{ g_0(X_0, u_0) + \min_{\substack{x_1,\dots,x_T \\ u_1,\dots,u_T}} \sum_{t=1}^{T} g_t(x_t, u_t) \;\text{ s.t. } \begin{cases} x_{t+1} = f_t(x_t, u_t) \\ x_1 = f_0(X_0, u_0) \end{cases} \right\} \\ &= \min_{u_0} \{ g_0(X_0, u_0) + V_1(x_1) \} \\ &= \min_{u_0} \{ g_0(X_0, u_0) + V_1(f_0(x_0, u_0)) \} \end{aligned}$$

We can see that the problems are nested, therefore we need to proceed in a **backward** way, starting from the last time step and going backward in time. In particular we need to proceed in the following order

1. solve the subproblem parametrized by x_1;
2. compute the value function V_1;
3. solve the outer problem
$$\min_{u_0} \{ g(X_0, u_0) + V_1(f_0(X_0, u_0)) \}$$

1.2 Dynamic Programming

We can now write the general DP algorithm, which is a backward procedure to compute the value function and the optimal policy. We start from the last time step and we proceed backward in time until we reach the initial time step.
At the last stage (T) the subproblem is trivial, since there are no more decisions to be taken, therefore we have

$$V_T(x_T) = g_T(x_T)$$

At time $T-1$ we compute the value function as

$$\min_{u_{T-1}} \{ g_{T-1}(x_{T-1}, u_{T-1}) + V_T(f_{T-1}(x_{T-1}, u_{T-1})) \}$$

In particular notice that in the formulation above we know the value of $V_T(x_T)$, therefore we can compute the value function at time $T-1$ and so on until we reach the initial time step. So at this moment we have an optimal policy for the time step $T-1$, that means that we know how to compute the optimal control action at time $T-1$ given the state at time $T-1$. We can repeat this procedure until we reach the initial time step, therefore we have an optimal policy for all time steps.
At time $T-2$ we compute $V_{T-2}(x_{T-2})$ as

$$\min_{u_{T-2}} \{ g_{T-2}(x_{T-2}, u_{T-2}) + V_{T-1}(f_{T-2}(x_{T-2}, u_{T-2})) \}$$

Notice that this value function $V_{T-2}(x_{T-2})$ is a function of the state x_{T-2}, therefore we have to compute it $\forall x_{T-2} \in \mathcal{X}_{T-2}$, which is generally a very hard task.

Problem Decomposition

We have seen that the optimal control problem is decomposed into **stage problems** that we can solve via **backward induction** :

$$V_t(x_t) = \min_{u_t}\{g_t(x_t, u_t) + V_{t+1}(f_t(x_t, u_t))\} \tag{2}$$

And $V_T(x_T) = g_T(x_T)$

A reasonable question at this point would be, in which cases is this stage problem formulation simple to solve ? We can list three scenarios:

- Decision space $\tilde{\mathcal{U}}$ is convec , or maybe finite.
- If g_t is convex in u_t.
- If V_{t+1} evaluated at the future step f_t is convex in u_t.

In practice, a very common setup in which the assumptions above are satisfied is when we have a **quadratic cost** g_t and a **linear dynamics** f_t, which is the so called **LQR problem**. Another way is also to use approximations of the value function, in which case maybe not all the assumptions above are satisfied, but we can still solve the stage problem.

1.3 Optimal LQR

The LQR problem is a special case of the optimal control problem, in which we have a linear dynamics and a quadratic cost. In particular we have:

- Markovian update and linear dynamics

$$x_{t+1} = A_t x_t + B_t u_t$$

- Quadratic cost function:

$$\sum_{t=0}^{T-1} \left(x_t^\top \, Q_t \, x_t + u_t^\top \, R_t \, u_t \right) + x_T^\top \, S \, x_T \quad \text{with } Q, S \geq 0 \, , \, R > 0$$

- Value function (i.e. minimum cost to go starting from x at time t) :

$$V_t(x) = \min_{u_t, \ldots, u_{T-1}} \sum_{k=t}^{T-1} \left(x_k^\top \, Q_k \, x_k + u_k^\top \, R_k \, u_k \right) + x_T^\top \, S \, x_T$$

$$\begin{aligned} \text{subject to } x_{k+1} &= A_k x_k + B_k u_k \quad \forall k = t, \ldots, T-1 \\ x_t &= x \end{aligned}$$

We say that the optimal cost is $V_0(x_0)$.

1.4 Dynamic Programming for LQR

Let us now specialize the dynamic programming recursion to the Linear Quadratic Regulator (LQR) problem. Recall that the value function represents the optimal *cost-to-go*, i.e. the minimum cost achievable from a given state when acting optimally from that point onward.

A key observation in the LQR setting is that the terminal cost is quadratic in the state. Indeed, from the problem formulation we have

$$V_T(x) = x^\top S x,$$

which is a convex quadratic function since $S \succeq 0$.

The main idea of the LQR derivation is to show that the value function $V_t(x)$ is also quadratic and has the form of $V_t(x) = x^\top P_t\, x$ for some symmetric positive semidefinite matrix P_t.
We will then show that the matrices P_t can then be computed recursively by working *backward in time* starting from the terminal time T. Eventually the the optimal control input u_t will be computed as a solution of a convex quadratic optimization problem, which admits a closed-form solution.

To derive the recursion, consider the dynamic programming stage problem at time t where the value function is

$$V_t(x) = \min_u \Big(x^\top Q\, x + u^\top R\, u + V_{t+1}(Ax + Bu) \Big).$$

We now proceed by induction, assuming that $V_{t+1}(x) = x^\top P_{t+1} x$ for some positive semidefinite matrix P_{t+1}. Then $V_t(x)$ can be written as

$$\begin{aligned} V_t(x) &= x^\top Q x + \min_u \Big(u^\top R u + (Ax + Bu)^\top P_{t+1}(Ax + Bu) \Big) \\ &= x^\top Q x + \min_u \Big(u^\top R u + x^\top A^\top P_{t+1} A x + 2u^\top B^\top P_{t+1} A x + u^\top B^\top P_{t+1} B u \Big) \\ &= x^\top Q\, x + \min_u \Big(u^\top (R + B^\top P_{t+1} B) u + x^\top A^\top P_{t+1} A x + 2u^\top B^\top P_{t+1} A x \Big). \end{aligned}$$

We set the gradient of the expression inside the minimization with respect to u to zero:

$$(R + B^\top P_{t+1} B) u + B^\top P_{t+1} A x = 0$$

Yielding the optimal control input

$$\boxed{u^* = -(R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x}$$

But now we need to check that $V_t(x)$ is indeed quadratic in X. That means $V_t(x) = x^\top P_t x$ for some P_t and we have to find how to compute P_t. We can do that by plugging the optimal control input u^* back into the expression for $V_t(x)$:

$$\begin{aligned} V_t(x) &= x^\top Q x + (u^*)^\top R u^* + (Ax + Bu^*)^\top P_{t+1}(Ax + Bu^*) \\ &= x^\top Q x + (u^*)^\top R u^* + x^\top A^\top P_{t+1} A x + 2(u^*)^\top B^\top P_{t+1} A x + (u^*)^\top B^\top P_{t+1} B u^* \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x + (u^*)^\top (R + B^\top P_{t+1} B) u^* + 2(u^*)^\top B^\top P_{t+1} A x \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x + (u^*)^\top \big((R + B^\top P_{t+1} B) u^* + 2B^\top P_{t+1} A x \big) \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x \\ &\quad -\big((R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x\big)^\top \big(-(R + B^\top P_{t+1} B)(R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x + 2B^\top P_{t+1} A x \big) \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x + -\big((R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x\big)^\top \big(B^\top P_{t+1} A x \big) \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x - x^\top A^\top P_{t+1} B (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x \\ &= x^\top \Big(Q + A^\top P_{t+1} A - A^\top P_{t+1} B (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A \Big) x \\ &= x^\top P_t x \end{aligned}$$

To complete the proof one should check that P_t is positive semidefinite. Here we homit the proof. ■

In cocnlusion we have been able to derive a closed-form expression for the optimal control input u^* for the LQR problem.Be aware that this solution is not valid anymore if the problem becomes constrained.

We summerize here the main results:

Optimal LQR solution

Backward induction

$$u_t = \underbrace{-(R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A}_{\Gamma_t} \, x_t$$

where

$$P_t = Q + A^\top P_{t+1} A - A^\top P_{t+1} B (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A \quad \text{and } P_T = S$$

Forward integration from x_0:

$$x_{t+1} = Ax_t + Bu_t$$

So clearly one could compute this whole open-loop sequence $u_0, \dots, u_{T-1}$ in a offline computation. But what if the optimal control input **u** takes us to a trajectory different from the one we compouted ? In this case we can use the optimal policy Γ_t to compute the optimal control input at each time step, given the current state x_t. This is the main advantage of having a closed-form solution for the optimal control input, since we can use it in a feedback way.

We can do a quick comparison of the computational cost of using these equations in a feedback way vs computing the whole open-loop sequence. By fixing m as the size of the states and n as the size of the control input, we have the following computational costs:

- **Offline computation**
  - $n \times n$ matrix multiplications to compute P_t for $t = T-1, \dots, 0$
  - $m \times m$ matrix inversions to compute Γ_t for $t = T-1, \dots, 0$
  - T iterations to compute P_t and Γ_t for $t = T-1, \dots, 0$
- **Online computation**
  - Storage of T $m \times n$ matrices Γ_t for $t = T-1, \dots, 0$
  - $m \times m$ matrix multiplications to compute u_t at each time step, given x_t

1.4.1 Infinite-Horizon LQR

In many practical control applications the system is required to operate over a very long time horizon. Examples include regulation problems where the controller must stabilize a system indefinitely, or tracking tasks where disturbances may occur continuously.

For these reasons it is natural to study the limit case where the horizon tends to infinity. The resulting problem is known as the **infinite-horizon Linear Quadratic Regulator (LQR)**.

Consider the linear time-invariant system

$$x_{t+1} = Ax_t + Bu_t, \qquad x_0 \text{ given}$$

and the infinite-horizon quadratic cost

$$\min_{u_0, u_1, \dots} \sum_{t=0}^{\infty} \left(x_t^\top Q x_t + u_t^\top R u_t \right), \qquad Q \succeq 0, \; R \succ 0.$$

Compared to the finite-horizon case, the cost now accumulates indefinitely. Therefore a first important question concerns the **feasibility** of the problem.

Feasibility

If the pair (A, B) is **stabilizable**, then there exists a control input sequence that yields a finite value of the infinite-horizon cost.

Indeed, if the system is stabilizable there exists a feedback matrix K such that the closed-loop matrix $A + BK$ has all eigenvalues inside the unit circle. Consider the linear feedback policy

$$u_t = Kx_t.$$

The state then evolves according to

$$x_t = (A + BK)^t x_0.$$

Substituting this into the cost yields

$$V(x_0) = \sum_{t=0}^{\infty} \left(x_t^\top Q x_t + x_t^\top K^\top R K x_t \right).$$

Using the closed-loop dynamics we obtain

$$V(x_0) = x_0^\top \left(\sum_{t=0}^{\infty} (A + BK)^{t\top} (Q + K^\top R K)(A + BK)^t \right) x_0.$$

Since $A + BK$ is stable, the above summation behaves like a *matrix geometric series* and therefore converges. Consequently the cost is finite.

But how do we compute the optimal policy for the infinite-horizon LQR problem ? The answer is that we can still apply the dynamic programming recursion, but now the value function does not depend on time. The solution is given by the **Discrete Algebraic Riccati Equation (DARE)**, which is a fixed-point equation for the matrix P that defines the value function.

Algebraic Riccati Equation

The iteration

$$P_{t-1} = Q + A^\top P_t A - A^\top P_t B (R + B^\top P_t B)^{-1} B^\top P_t A \qquad P_0 = 0$$

converges, for $t \to -\infty$, to a unique positive semidefinite solution $P_\infty \succ 0$ of the algebraic Riccati equation

$$P = Q + A^\top P A - A^\top P B (R + B^\top P B)^{-1} B^\top P A$$

We can show that then the optimal feedback control in the infinite-horizon case is given by

$$u_t = \underbrace{-(R + B^\top P B)^{-1} B^\top P A}_{F_\infty} x_t$$

and thus minimizes the infinite horizon cost and yields $V(X_0) = X_0^\top P X_0$.
We now show again a comparison of the computational cost of using the infinite-horizon solution in a feedback way vs computing the whole open-loop sequence. By fixing m as the size of the states and n as the size of the control input, we have the following computational costs:

- **Offline computation**

  - $n \times n$ matrix multiplications to compute P
  - $m \times m$ matrix inversions to compute F_∞
  - T iterations to compute P and F_∞ for $t = T-1, \dots, 0$
  - "Infinite iterations" to compute P for $t \to -\infty$[1]

- **Online computation**
  - Storage of a **single** $m \times n$ matrix F_∞. So less memory is required compared to the finite-horizon case, since we do not need to store T matrices Γ_t.
  - $m \times n$ matrix multiplications to compute u_t at each time step, given x_t

A natural question at this point is whether the optimal infinite-horizon feedback law is also *stabilizing*. Indeed, even if the infinite-horizon cost is well defined and the Riccati equation admits a solution, one would still like to know whether the resulting closed-loop dynamics

$$x_{t+1} = (A + BF_\infty)x_t$$

are asymptotically stable.

This is a meaningful question because the quadratic cost does *not* necessarily penalize all state directions. In particular, if some unstable state component is not "seen" by the cost, then the optimizer may have no incentive to stabilize it.

Illustrative example Consider the system

$$x_{t+1} = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} x_t + u_t, \qquad Q = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \qquad R = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Here both open-loop modes are unstable, since the eigenvalues of A are 2 and 3. However, the stage cost penalizes only the first state component x_1, while the second component x_2 does not appear in the state cost. Therefore, from the viewpoint of the objective function, an unstable behavior in the second state may go completely unnoticed unless it is indirectly forced to be controlled through the dynamics.

This already suggests an important principle:

> Unstable modes must be visible in the cost function, otherwise the optimal controller may fail to stabilize them.

To make this statement precise, write $Q = C^\top C$. Then the cost penalizes exactly the directions that are observable through the pair (C, A).

Stability of the optimal infinite-horizon LQR controller

Let $Q = C^\top C$. The optimal infinite-horizon feedback F_∞ stabilizes the system if and only if

1. the pair (A, B) is stabilizable;
2. the pair (C, A) has no unobservable unstable modes.

Equivalently, all unstable modes of A must be detectable through the state penalty Q.

The second condition is often stated by saying that the pair $(A, Q^{1/2})$ must be **detectable**. Detectability is weaker than observability: it does not require all modes to be observable, but only the unstable ones.

[1] P_∞ can be computed by solving the Algebraic Riccati Equation

Interpretation This theorem has a clear practical meaning. The matrix R penalizes the control effort, while Q determines which state directions the controller tries to regulate. If an unstable mode is not penalized by Q, then the optimization problem may prefer not to spend control effort on that mode, since doing so does not improve the objective. As a consequence, the optimal controller may exist but fail to stabilize the closed-loop system.

Hence, in practice:

- (A, B) stabilizable ensures that unstable modes can in principle be controlled;
- detectability of $(A, Q^{1/2})$ ensures that every unstable mode matters in the cost.

When both conditions hold, the stabilizing solution P of the algebraic Riccati equation yields the optimal feedback

$$u_t = F_\infty x_t, \qquad F_\infty = -(R + B^\top P B)^{-1} B^\top P A,$$

and the closed-loop matrix $A + BF_\infty$ has all eigenvalues inside the unit disk.

2 Model Predictive Control

2.1 Introduction

In the previous chapter we have seen an overview of the LQR problem and how to solve it using dynamic programming. In that formulation, we assumed that there were no constraints on the state and control input. However, in many real-world applications, constraints on the state and input must be explicitly taken into account.
In the presence of constraints, the problem cannot, in general, be solved in closed form via backward induction as in the unconstrained LQR case, since the value function is no longer quadratic.The resulting optimization problem becomes:

Constrained LQ finite horizon problem

$$\begin{aligned} \min_{u_0,\dots,u_{T-1},x_0,\dots,x_T} & \sum_{t=0}^{T-1} \left(x_t^\top Q x_t + u_t^\top R u_t \right) + x_T^\top S x_T \\ \text{s.t.} \quad & x_{t+1} = A x_t + B u_t, \quad t = 0, \dots, T-1 \\ & x_0 = x_0 \\ & x_t \in \mathcal{X}_t \\ & u_t \in \mathcal{U}_t \end{aligned}$$

Usually, the constraint sets are convex, e.g., polytopes.

One could solve this optimization problem in a **one-shot** fashion (i.e., open-loop strategy), obtaining an optimal input sequence $u_0, \dots, u_{T-1}$ for the given initial state x_0, instead of a state-feedback policy as in the unconstrained case (where $u_t = \Gamma_t x_t$).
However, if disturbances or model mismatch cause the system to deviate from the predicted trajectory, the previously computed control sequence is no longer optimal. As a result, the system will follow a different trajectory than expected (see Figures 1 and 2).
To address this issue, the optimal control problem must be re-solved using the updated state information. Since we assume a Markovian system, the current state contains all the information needed to compute an optimal control sequence for the future. This implies that the optimization problem must be solved repeatedly, at each time step, within one sampling interval.

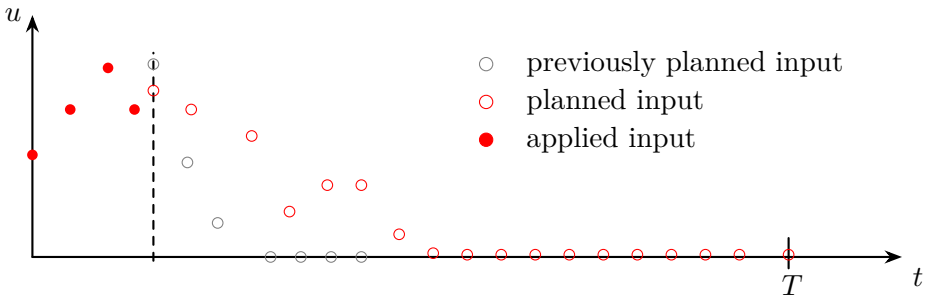

Figure 1: Receding-horizon update of the input sequence: previously planned inputs are shifted, a new optimal input sequence is computed based on the current state.

The idea that emerges from this discussion is to solve the optimization problem in a **receding horizon** fashion. At each time step, we solve a finite-horizon optimal control problem of length T, and apply only the first control input of the optimal sequence (see Figure 3).
At the next time step, the new state is measured, and the optimization problem is solved again

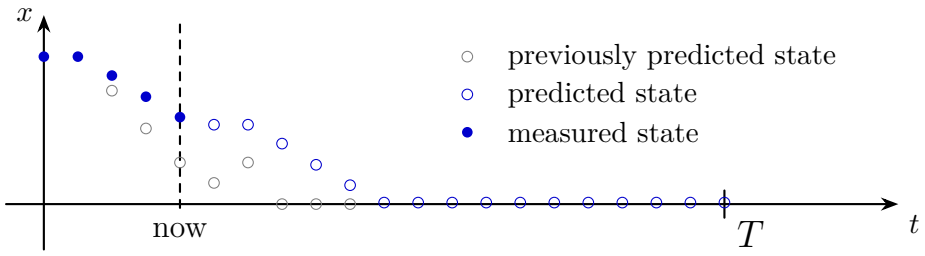

Figure 2: Receding-horizon update of the state trajectory: previously predicted states are corrected using the measured state at the current time, and a new optimal state trajectory is predicted over the horizon.

with the updated initial condition. If the system is time-invariant, the optimization problem remains the same at each time step, except for the initial state. The corresponding predicted state evolution over the horizon is illustrated in Figure 4.

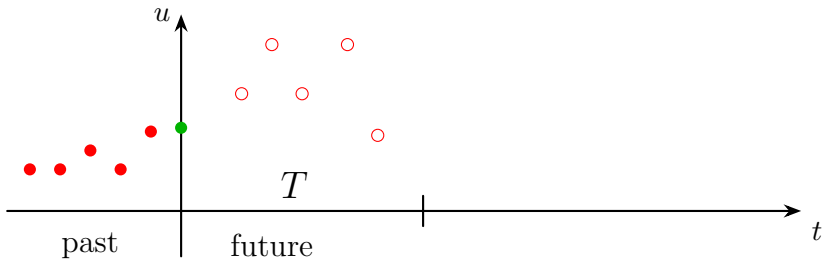

Figure 3: Finite-horizon input sequence in receding horizon control: at the current time, an optimal sequence of future inputs over a horizon of length T is computed, while past inputs are fixed.

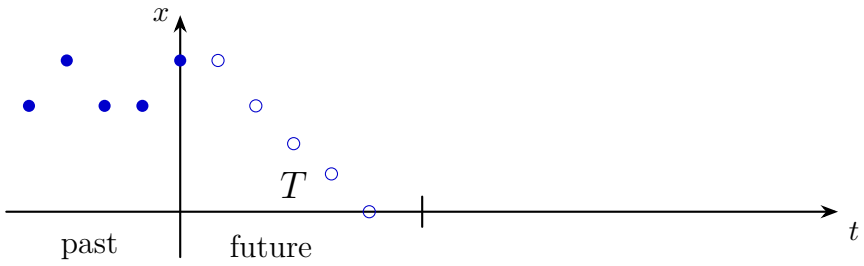

Figure 4: Finite-horizon state prediction in receding horizon control: starting from the current state, future states are predicted over a horizon of length T using the planned input sequence.

Thus the following scheme defines the receding horizon control strategy, also known as **Model Predictive Control (MPC)**:

1. At time t, measure the current state x_t.
2. Solve the finite-horizon optimal control problem with initial state x_t to obtain the optimal input sequence $u_t, u_{t+1}, \ldots, u_{t+T-1}$.
3. Apply the first control input u_t to the system.
4. Increment time $t \leftarrow t + 1$ and repeat from step 1.

The resulting **open-loop optimal control problem** corresponds to a *parametric optimization problem* parametrized in the initial state x.

Parametric optimization problem

$u^*(x)$ is determined by[a]

$$\begin{aligned} \min_{\mathbf{u},\mathbf{x}} \quad & \sum_{k=0}^{T-1} \left(x_k^\top Q x_k + u_k^\top R u_k \right) + x_T^\top S x_T \\ \text{s.t.} \quad & x_{k+1} = A x_k + B u_k \\ & x_0 = x \\ & x_k \in \mathcal{X}_k \\ & u_k \in \mathcal{U}_k \end{aligned}$$

[a] Notation: k is the index for the optimization problem, while t is the index for the time steps of the system.

The parametric nature of the problem immediately raises the question of whether the resulting control strategy is *feedforward* or *feedback*. Although the optimization problem computes an open-loop input sequence, the receding horizon implementation introduces feedback implicitly. Indeed, at each time step the optimization problem is solved using the current state x_t, and only the first input of the optimal sequence is applied. Therefore, the implemented control law can be written as

$$u_t = u^*(x_t),$$

which defines a **state-feedback control law**. One can have a visual intuition of this feedback structure by looking at the block diagram in Figure 5.

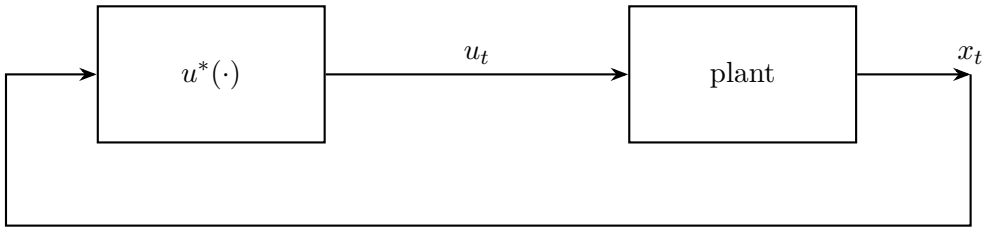

Figure 5: Block diagram of the MPC control scheme

We can establish some properties about this optimization problem:

- Under **continuity** assumptions on the cost and the dynamics, the minimum cost is a constinuous function of the initial state x.
- Under **compactness** assumptions on the constraints, the parametric optimization problem admits a solution for all x in the feasible set.
- The map $x \mapsto u^*(x)$ is not necessarily continuos for a general cost function

In the case of linear dynamics, quadratic cost, and convex constraints, the optimization problem is a convex quadratic program (QP) parametrized in the initial state x. In this case, the map $x \mapsto u^*(x)$ is piecewise affine and continuous. However, in general, the control law induced by MPC can be nonlinear and non-smooth.

MPC control law

As a consequence, MPC can be interpreted as a **memory-less, nonlinear, time-invariant feedback control law**. The nonlinearity arises from the presence of constraints, even when the system dynamics are linear and the cost is quadratic.

A natural question is whether MPC can achieve *perfect tracking*, i.e., zero steady-state error. In general, this is not guaranteed without additional design choices (e.g., terminal ingredients or disturbance modeling), since the controller repeatedly solves a finite-horizon problem and does not explicitly enforce asymptotic properties beyond the prediction horizon. To better understand the relation between MPC and the finite-horizon optimal control problem, consider the following example:

Example

$$\begin{aligned} \min_{\mathbf{x},\mathbf{u}} \quad & \sum_{t=0}^{T-1} a^t |u_t| + |x_T|^2, \quad |a| < 1 \\ \text{s.t.} \quad & x_{t+1} = x_t + u_t. \end{aligned}$$

Two different questions can be asked:

1. What is the optimal open-loop solution $(\mathbf{x}^*, \mathbf{u}^*)$?
2. What is the MPC feedback law $x \mapsto u^*(x)$ obtained by solving the problem in a receding horizon fashion?

To solve this we observe that the cost is not quadratic and that the cost of applying inouts is discounted over time. Furthermore the state is penalized only at the end of the horizon.
Thus the optimal solution is to simply wait for the last time step and then apply $u_T = -x_T$, which yields a cost of $|x_0|^2$.
In contrast, the MPC feedback law is given by the control law $x \mapsto u^*(x) = 0$.
A key observation is that the optimal trajectory associated with the finite-horizon problem is, in general, **never realized in closed loop**, even in the absence of disturbances or model mismatch. This is because at each time step the optimization problem is re-solved with a shifted horizon, leading to a different optimal sequence than the one computed previously. ■

Consider now the unconstrained linear-quadratic problem:

$$\begin{aligned} \min_{\mathbf{x},\mathbf{u}} \quad & \sum_{t=0}^{T-1} \left(x_t^\top Q x_t + u_t^\top R u_t \right) + x_T^\top S x_T, \quad Q, S \succeq 0,\ R \succ 0 \\ \text{s.t.} \quad & x_{t+1} = A x_t + B u_t. \end{aligned}$$

Finite-time LQR yields a time-varying linear feedback law

$$\begin{aligned} u_t &= \Gamma_t x_t \\ \Gamma_t &= -(R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A \end{aligned}$$

where the matrices P_t are computed backward in time. The control gain explicitly depends on time through P_{t+1}.
In contrast, **MPC** repeatedly solves the same finite-horizon problem at each time step. As a result, the applied control law is

$$u_t = \Gamma_0 x_t,$$

where Γ_0 is the gain associated with the first step of the horizon. Hence, MPC induces a **linear time-invariant** feedback law.
More explicitly, at time $t = 0$ both approaches yield the same control input:

$$u_0^{\text{MPC}} = -(R + B^\top P_1 B)^{-1} B^\top P_1 A x_0 = u_0^{\text{LQR}}.$$

However, at the next time step a discrepancy appears. Finite-time LQR applies

$$u_1^{\text{LQR}} = -(R + B^\top P_2 B)^{-1} B^\top P_2 A x_1,$$

while MPC recomputes the problem and applies again

$$u_1^{\text{MPC}} = -(R + B^\top P_1 B)^{-1} B^\top P_1 A x_1.$$

Therefore, even in the unconstrained case, MPC does not reproduce the finite-horizon optimal control sequence beyond the first step.

2.2 Implementation of MPC

The implementation of MPC requires solving an optimization problem at each time step. We show two main approaches to implement MPC:

1. Compute the **function** $u^*(\cdot)$ **offline**, and then evaluate it online at each time step.
   - It is extremely complex to solve with exception of very few cases
   - Abundant offline computational resources are required
   - Large memory is required to store the function $u^*(\cdot)$
2. Compute the **value** of the function $U^*(\cdot)$ **online** at each time step by solving the optimization problem.
   - It is ogten a tractable optimization problem.
   - It has limited time to perform computation online.

2.2.1 Explicit MPC

Explicit MPC corresponds to the first implementation strategy: instead of solving the optimization problem online at each sampling instant, we try to compute the feedback law$x \mapsto u^*(x)$ offline.
We have already seen one important case. In the **unconstrained linear MPC** case, the optimizer is simply the first element of the finite-horizon LQR solution. Therefore, the applied input can be written as

$$u_t = u^*(x_t) = \Gamma_0 x_t,$$

so the control law is linear.

The natural question is what happens when *constraints* are introduced. Consider the standard constrained linear MPC problem

Linear MPC with linear constraints

$$\begin{aligned}
\min_{\mathbf{u},\mathbf{x}} \quad & \sum_{k=0}^{T-1} \left(x_k^\top Q x_k + u_k^\top R u_k \right) + x_T^\top S x_T \\
\text{s.t.} \quad & x_{k+1} = A x_k + B u_k, \qquad k = 0, \dots, T-1, \\
& x_0 = x, \\
& x_k \in \mathcal{X}, \qquad \mathcal{X} = \{x \in \mathbb{R}^n : Cx \le d\}, \\
& u_k \in \mathcal{U}, \qquad \mathcal{U} = \{u \in \mathbb{R}^m : Eu \le f\},
\end{aligned}$$

with $Q, S \succeq 0, \qquad R \succ 0.$
This is the most common MPC formulation: linear dynamics, quadratic cost, and polyhedral state/input constraints.

This problem is a **convex quadratic program** parametrized by the initial state x. In general, the optimizer is no longer globally linear. However, due to the quadratic cost and linear

constraints, it still has a very specific structure: the explicit MPC law turns out to be **piecewise affine**. The idea is best understood through a very simple example.

Example Consider a scalar integrator with horizon $T = 1$:

$$\begin{aligned} \min_{u_0, x_1} \quad & u_0^2 + x_1^2 \\ \text{s.t.} \quad & x_1 = x_0 + u_0, \\ & x_1 \leq 1. \end{aligned}$$

This example is useful because:

- it is a quadratic problem with a linear constraint,
- it is parametrized by the current state x_0,
- the equality constraint allows us to eliminate the variable x_1,
- the problem is convex, hence the KKT conditions are necessary and sufficient.

Using the model equation $x_1 = x_0 + u_0$, we can eliminate x_1 and rewrite the problem as

$$\min_{u_0} f(u_0) \qquad \text{s.t.} \qquad g(u_0) \leq 0,$$

where

$$\begin{aligned} f(u_0) &= u_0^2 + (x_0 + u_0)^2, \\ g(u_0) &= x_0 + u_0 - 1. \end{aligned}$$

The derivatives are

$$\begin{aligned} \nabla f(u_0) &= 4u_0 + 2x_0, \\ \nabla g(u_0) &= 1. \end{aligned}$$

KKT conditions for one inequality constraint

For a problem of the form

$$\min_{x} f(x) \qquad \text{s.t.} \qquad g(x) \leq 0,$$

the KKT conditions are

$$\begin{aligned} \nabla f(x) + \mu \nabla g(x) = 0, \qquad & \mu \geq 0, \\ g(x) \leq 0, & \\ \mu\, g(x) = 0. & \end{aligned}$$

The last relation is the *complementary slackness* condition.

Applying the KKT conditions to the problem gives

$$\begin{aligned} 4u_0 + 2x_0 + \mu = 0, \qquad & \mu \geq 0, \\ x_0 + u_0 - 1 \leq 0, & \\ \mu(x_0 + u_0 - 1) = 0. & \end{aligned}$$

The complementary slackness condition implies that two cases are possible.

Case 1: inactive constraint. Assume first that the inequality is inactive, so

$$\mu = 0.$$

Then stationarity becomes

$$4u_0 + 2x_0 = 0,$$

hence

$$u_0 = -\frac{x_0}{2}.$$

To check whether this candidate is feasible, we substitute it into the constraint:

$$x_0 + u_0 - 1 = x_0 - \frac{x_0}{2} - 1 = \frac{x_0}{2} - 1.$$

Therefore feasibility requires

$$\frac{x_0}{2} - 1 \leq 0 \qquad \Longleftrightarrow \qquad x_0 \leq 2.$$

So, for all initial states satisfying $x_0 \leq 2$, the optimal input is

$$u_0^* = -\frac{x_0}{2}.$$

Case 2: active constraint. Assume now that the inequality is active, so

$$x_0 + u_0 - 1 = 0.$$

Then

$$u_0 = 1 - x_0.$$

Substituting this into the stationarity condition,

$$4u_0 + 2x_0 + \mu = 0,$$

we obtain

$$\mu = -4u_0 - 2x_0 = -4(1 - x_0) - 2x_0 = 2x_0 - 4.$$

Since $\mu \geq 0$, this case is valid only if

$$2x_0 - 4 \geq 0 \qquad \Longleftrightarrow \qquad x_0 \geq 2.$$

Hence, for all $x_0 \geq 2$, the optimal input is

$$u_0^* = 1 - x_0.$$

Combining the two cases, the explicit solution is

Explicit solution of the toy MPC problem

$$u_0^*(x_0) = \begin{cases} -\dfrac{x_0}{2}, & x_0 \leq 2, \\ 1 - x_0, & x_0 \geq 2. \end{cases}$$

This example shows the main structural property of explicit MPC:

- the control law is **continuous**,
- the control law is **piecewise affine**,
- each affine expression is valid in a region where the same set of constraints is active.

The reason is the following. Once the active constraints are known, the KKT conditions reduce to a linear system in the decision variables and the multipliers. Solving that linear system gives an optimizer that depends *affinely* on the parameter x. When the active set changes, a different affine expression is obtained. Therefore, the state space is partitioned into regions, and on each region the controller is affine.

Although the explicit solution is conceptually appealing, it becomes difficult to use in large problems. In practice:

- there may be tens or hundreds of constraints,
- the number of regions may become very large,
- the offline computation required to determine all regions can be very long,
- even online, one must determine in which region the current state lies before evaluating the affine law.

As a consequence, explicit MPC is mainly attractive when the system dimension is small and the sampling time is so short that solving a quadratic program online would be too expensive. For larger systems, the most common approach is instead to solve the MPC optimization problem online at each sampling instant.

2.3 Stability

A useful way to think about MPC is as a *feedback design tool*. Once the prediction horizon T and the weights Q, R, S are chosen, the receding-horizon implementation induces a control law of the form

$$u_t = u^*(x_t).$$

Even though the optimization problem computes an open-loop input sequence, only the first input is applied and the problem is solved again at the next time step. Therefore, the implemented controller is a **static, time-invariant, nonlinear feedback law**. The nonlinearity does not necessarily come from the dynamics; it already appears because of the repeated constrained optimization.
From this perspective, the main question is no longer only how to solve the optimization problem, but whether the resulting closed-loop system is stable. There are two conceptually different ways to approach this question. The first is an *analysis* point of view: one tries to compute the control law $u^*(\cdot)$ explicitly and then study the stability of the closed-loop dynamics. In general, this is difficult, because the MPC law can be piecewise affine or more generally nonlinear and non-smooth. The second is a *design* point of view: instead of computing the feedback law explicitly, one looks for conditions on the design parameters T, Q, R, S that guarantee closed-loop stability directly. This is the more useful viewpoint in practice.

To study stability, the standard tool is Lyapunov analysis. Consider a discrete-time system

$$x_{t+1} = f(x_t),$$

and suppose that the origin $x = 0$ is an equilibrium, that is, $f(0) = 0$.

Stable equilibrium

The equilibrium $x = 0$ is said to be **stable** if

$$\forall \varepsilon > 0,\ \exists \delta > 0 \text{ such that } \|x_0\| < \delta \implies \|x_t\| < \varepsilon \quad \forall t \geq 0.$$

In words, if the initial condition is sufficiently close to the equilibrium, then the entire trajectory remains close to it for all future times.

Stability alone only guarantees that trajectories do not move far away from the equilibrium. A stronger notion is asymptotic stability, which additionally requires convergence to the equilibrium.

Asymptotically stable equilibrium

The equilibrium $x = 0$ is **asymptotically stable** if it is stable and

$$\lim_{t\to\infty} \|x_t\| = 0.$$

Hence, trajectories starting sufficiently close to the origin remain close to it and eventually converge to it.

The classical criterion to prove asymptotic stability is based on a Lyapunov function.

Lyapunov theorem for discrete-time systems

Assume there exists a real-valued function $W : \mathbb{R}^n \to \mathbb{R}$ such that

$$\begin{aligned} W(0) &= 0, \\ W(x) &> 0 \qquad \forall x \neq 0, \\ W(f(x)) &< W(x) \qquad \forall x \neq 0. \end{aligned}$$

Then the equilibrium $x = 0$ is asymptotically stable.

The idea is simple: W plays the role of an energy-like quantity. It is zero only at the equilibrium, positive everywhere else, and strictly decreases along the closed-loop trajectories. Therefore, the system is forced to move toward the origin.

This Lyapunov viewpoint explains why stability is relatively transparent for *infinite-horizon* MPC. Suppose that, at time t, the controller computes an input sequence that minimizes the infinite-horizon cost

$$V_t^\infty(x, \mathbf{u}) = \sum_{k=t}^{\infty} g_k(x_k, u_k),$$

subject to the system dynamics. If the minimum is attained, it is natural to define the optimal value function

$$W(x) = \min_{\mathbf{u}} V_t^\infty(x, \mathbf{u}).$$

Now comes the key observation. By Bellman's principle of optimality, if an input sequence is optimal from time t, then its tail is optimal from time $t + 1$ onward. In other words, after applying the first optimal input $u^*(x_t)$ and reaching the next state x_{t+1}, the remaining part of the optimal sequence is still optimal for the optimization problem starting at $t + 1$. As a consequence, along the closed-loop trajectory one obtains

$$W(x_{t+1}) = W(x_t) - g_t(x_t, u^*(x_t)).$$

Therefore, if the stage cost satisfies suitable positivity properties, for example if

$$g_t(x_t, u_t) > 0 \qquad \text{for all } x_t \neq 0,$$

then

$$W(x_{t+1}) < W(x_t) \qquad \text{for all } x_t \neq 0.$$

This means that the optimal value function is a natural Lyapunov candidate for the closed-loop system. Under reasonable assumptions, it follows that the origin is asymptotically stable.

This argument is very elegant, but it also hides some subtleties. First, an infinite-horizon MPC problem is infinite-dimensional, so from a computational viewpoint it is not directly implementable as stated. Second, one is implicitly assuming that the global minimum exists and is attained. Third, the stage cost must satisfy suitable properties so that the value function is positive definite and decreases strictly along the closed loop. These requirements are analogous in spirit to the assumptions encountered in infinite-horizon LQR.

The situation changes significantly for *finite-horizon* MPC. In that case, the same Lyapunov argument does not apply automatically. At time t, the controller minimizes a cost over the interval $[t, t+T]$. At time $t+1$, however, the optimization problem is not simply the tail of the previous one: the first stage cost has disappeared, but a new term has entered at the end of the horizon. Therefore, the controller at time $t+1$ is minimizing a *different* objective. As a result, the optimizer changes, and the finite-horizon value function is not automatically decreasing along the closed-loop trajectory.

This is the fundamental reason why closed-loop stability is immediate for infinite-horizon optimal control, but not for finite-horizon receding-horizon MPC. For finite-horizon MPC, stability must be enforced through additional design choices. This will be the starting point for the next part of the discussion.